



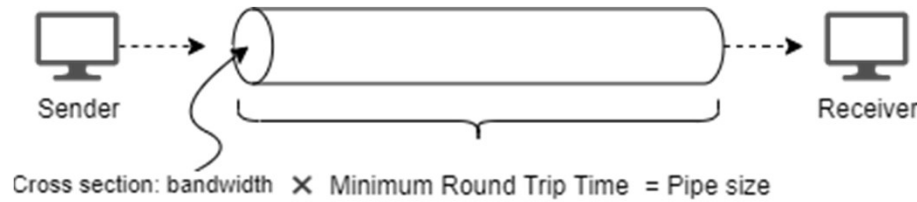
Samueli
Computer Science

Exploring Rate-Based Congestion Control in NDN

Sichen Song, Lixia Zhang
UCLA Computer Science

Problem description

- Congestion control algorithms in TCP/IP are window-based in general.
 - Estimating “pipe size” and adjusting congestion window to be the pipe size.



- In NDN, dynamic multipath forwarding and caching are introduced
 - Pipe size concept is no longer applicable
- Thus, a congestion control algorithm for NDN should:
 - R1: adaptive to the available bandwidth over all active paths.
 - R2: be insensitive to RTT variations.

Design goals

Let end consumers do measurement and adjust transmission rate.

Four goals in importance order

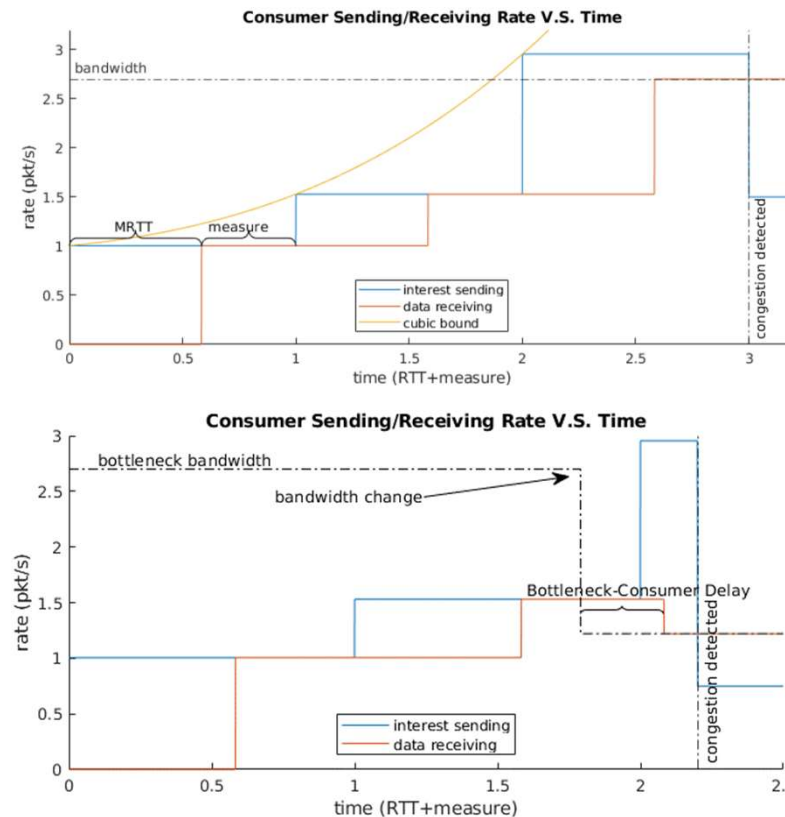
- Prevent long queue building up
- High bandwidth usage
- Do not rely on packet loss as congestion signal
- Good fairness (at least no starvation)

Report of our work-in-progress design

1. Our current rate-based congestion control design
2. Things we learned through the design and evaluation process
 1. Measurement interference between consumers under caching

Rate based congestion control design

- **Control the interest sending rate** instead of congestion window size
- Each round, start by sending interest at same rate for the maximum round trip time (MRTT) among forwarding paths.
- Then, **measure the data arrival rate**.
- Detect congestion by
 - Thresholding the difference between data arrival rate and the current interest sending rate. If sending is faster than arrival, there's congestion.
 - Detect drop of data arrival rate, which indicates a congestion caused by external factors.
- Adjust the interest sending rate accordingly
 - No congestion: increase the interest sending rate according to a CUBIC curve
 - Has congestion: multiplicatively decrease the interest sending rate.

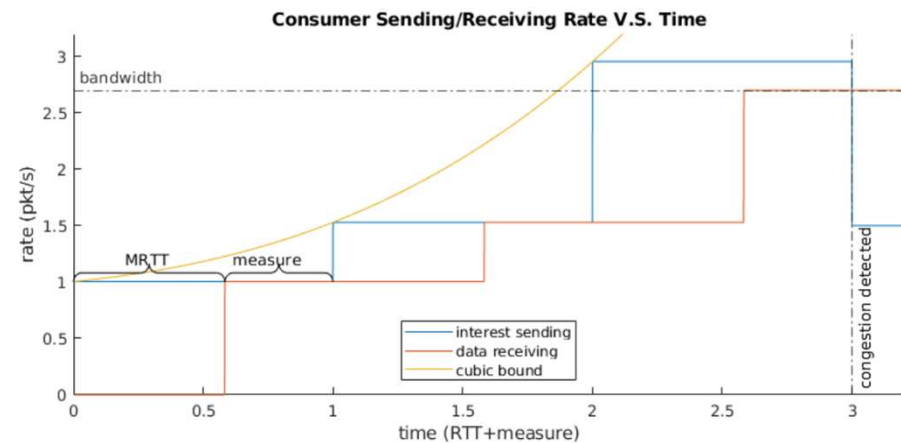


MRTT and measurement period

RTT and measurement time serve two purposes

- Wait for MRTT to observe result of previous adjustment.
- Measure data arrival rate for “measure time” to reduce noise.

A good measurement period depend on network dynamics instead of RTT. Changing measure time enables trade-off between accuracy and latency.

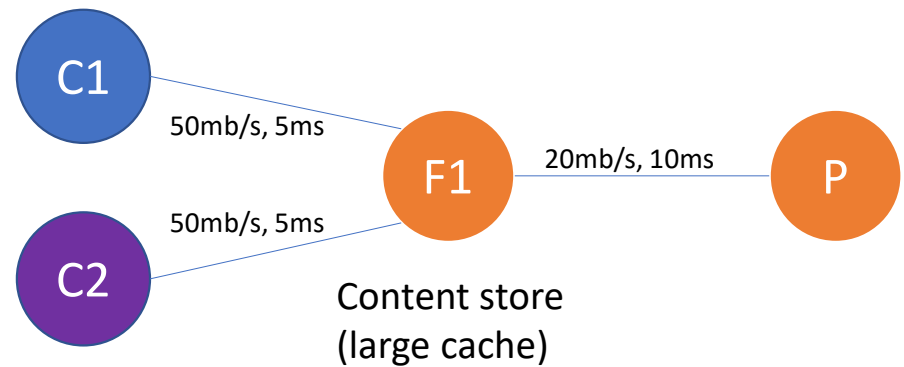


Report of our work-in-progress design

1. Our current rate-based congestion control design
2. Things we learned through the design and evaluation process
 1. Measurement interference between consumers under caching

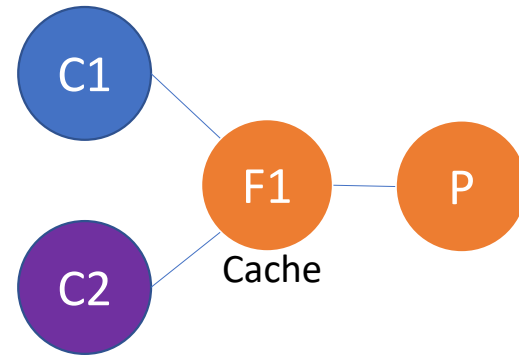
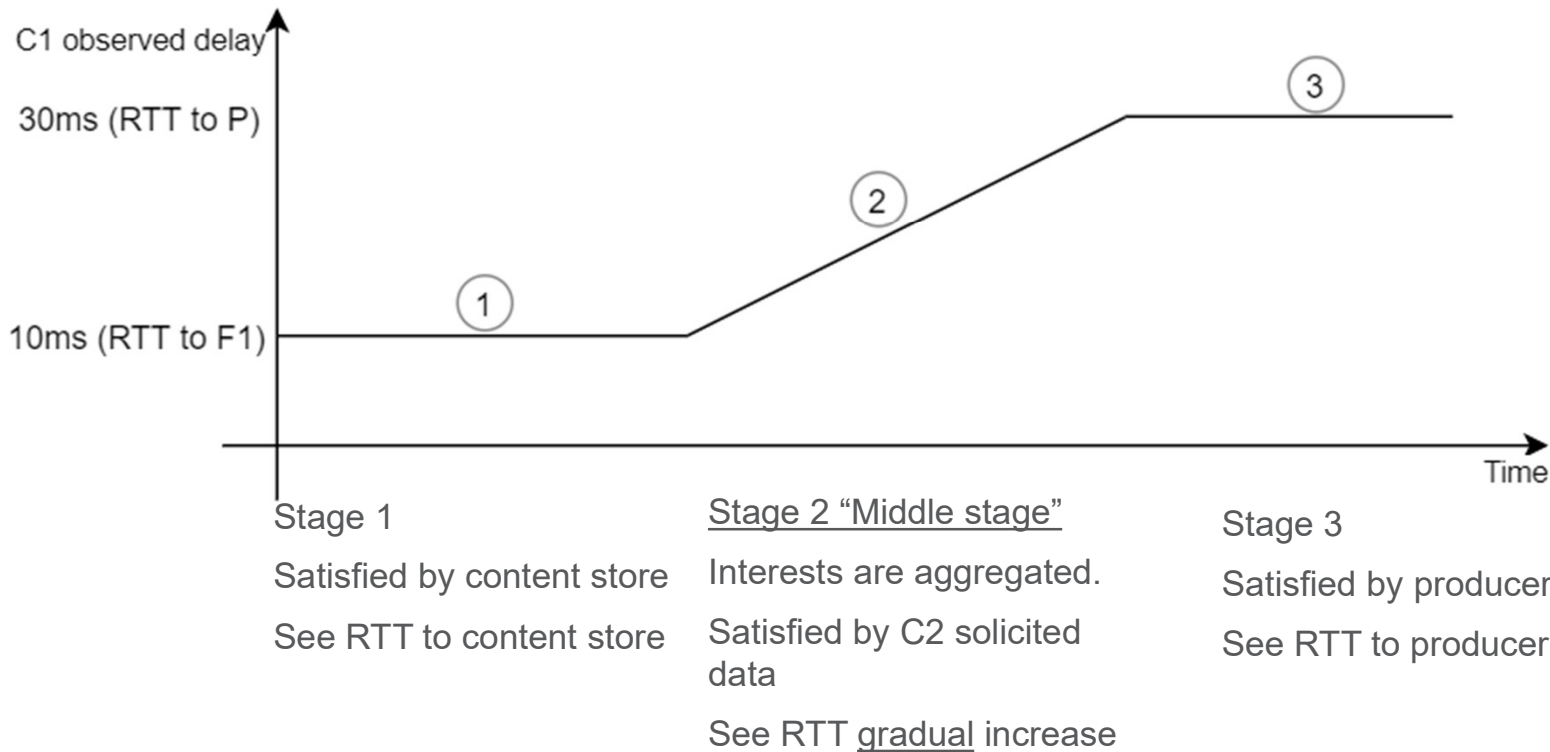
Interplay between consumers under caching

- Consumers C1 and C2 fetch the same object in order.
- C2 started early and made more progress.
- Forwarder F1 has a large content store.
- Expected: C1 will catch up with C2 and be satisfied by producer

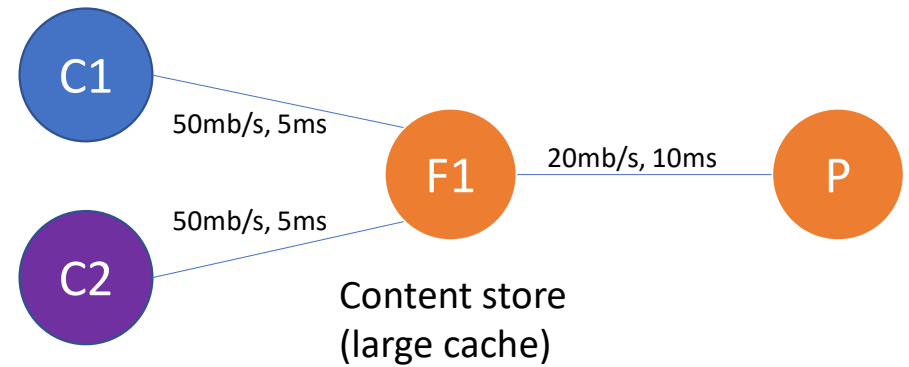
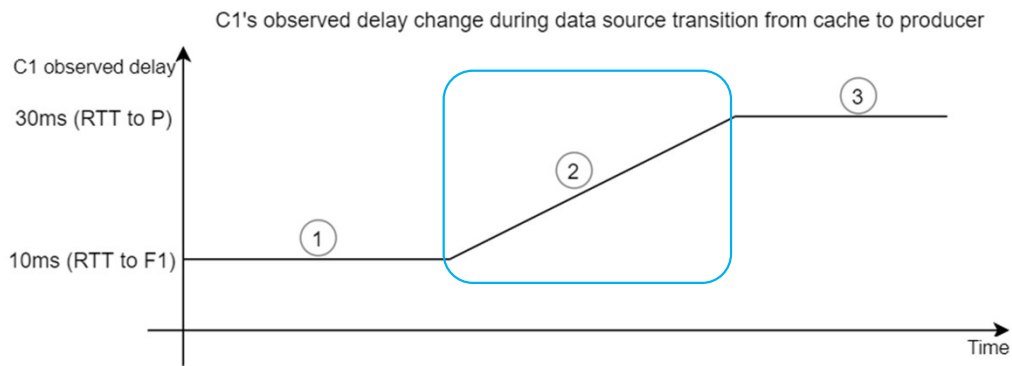


c1's RTT change under transition of data source

C1's observed delay change during data source transition from cache to producer



False congestion of the “middle stage” stage



Stage 2 “Middle stage”

Interests are aggregated.

Satisfied by C2 solicited data

RTT gradual increase

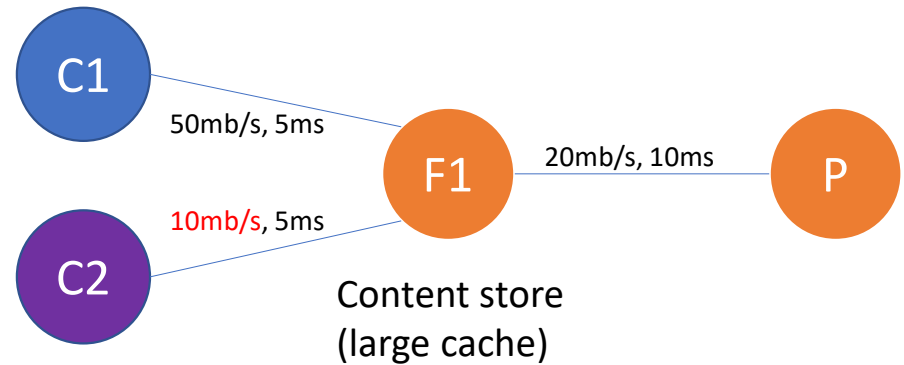
For C1

1. interest sending is faster than data arrival
 - C1's data arrival rate is C2's interest sending rate
2. gradual RTT increase

1&2 => congestion

Rate limiting due to false congestion

- The “false congestion” will be triggered whenever C1 is catching up
- Thus, C2’s bottleneck may rate limit C1
 - Even if C1 is not using that link
- This issue illustrates the failure of being tolerant to RTT variance (R2) of our initial design.
- The issue may also show up with existing congestion control that rely on MIN_RTT such as TCP Vegas or BBR



Solutions to rate limiting effect

- Two potential solutions
 - Let C1 detect the “middle stage”:
 - Once seeing RTT increase, C1 reorder its requests. Due to C2’s different segment requesting order, C1 will see data reordering or RTT sample jittering on the reordered segments.
 - Let forwarders provide queueing delay information. If C1 observe RTT increase/rate mismatch without queueing delay increase, it will not give up bandwidth.

Future work

- Explore some non-Cubic methods to control interest sending.
- Study the algorithm behavior under caching and multipath forwarding. Identify and address issues in these cases.
- Explore potential forwarder feedbacks that benefits the performance

Thank You
